ServiHogar Durango

Documentación técnica y académica de la versión final integrada

Castañón Galván Edgar Ailton

Pulgarin Flores Yosef Enrique

Ramírez Campos Jesús Alejandro

Asignatura: Programación / Desarrollo Web

Docente: Rubén Pizarro Gurrola

Grupo: 8Y — 7.º semestre

Fecha: octubre de 2026

Documento de trabajo basado en la documentación técnica proporcionada por el equipo. Los resultados de despliegue deben validarse con pruebas reales.

Resumen

ServiHogar Durango es una propuesta de plataforma web para solicitar y administrar servicios domésticos de plomería, electricidad y limpieza. La documentación previa del equipo describe una aplicación construida con Flask y PostgreSQL, con despliegue previsto en Render y base de datos alojada en Aiven. Esta versión de mejora reúne una interfaz visual consistente, flujos de solicitud y seguimiento, autenticación por roles y una guía de operación. La validación definitiva depende de ejecutar pruebas con la configuración real de nube y los datos del equipo.

Palabras clave: desarrollo web, Flask, PostgreSQL, experiencia de usuario, despliegue en la nube, gestión de servicios.

1. Introducción

La plataforma busca facilitar el contacto entre clientes y especialistas para resolver necesidades del hogar. El sistema propone un catálogo público, un formulario para registrar solicitudes, un portal para consultar el avance y un espacio administrativo para supervisar el flujo operativo. El rediseño prioriza legibilidad, navegación clara, formularios consistentes y adaptación a pantallas móviles.

2. Antecedentes y alcance del anteproyecto

La documentación entregada por el equipo señala siete tablas principales en el diseño inicial: usuarios, perfiles, servicios, especialistas_habilidades, solicitudes_servicio, historial_estatus y reseñas. También documenta autenticación, asignación de especialistas, panel de cliente, panel de especialista y dashboard ejecutivo. Los pagos en línea, las notificaciones por SMS/WhatsApp y la facturación electrónica se mantienen fuera del alcance de esta etapa.

3. Objetivos

Objetivo general

Consolidar una versión integrada de ServiHogar Durango que mejore la experiencia visual y organice los principales flujos de solicitud, asignación y seguimiento de servicios, junto con documentación reproducible para su instalación y despliegue.

Objetivos específicos

- Definir una identidad visual consistente y una interfaz adaptable a escritorio y móvil.
- Integrar validación de formularios y persistencia de solicitudes en base de datos.
- Separar las vistas y operaciones por rol de usuario en el servidor.

- Documentar la configuración de GitHub, Render y Aiven sin exponer secretos.
- Establecer una lista de pruebas para verificar los flujos antes de la entrega académica.

4. Metodología y evolución de los entregables

La documentación original organiza el trabajo en tres entregables: análisis y diseño, desarrollo de la lógica central, e integración y despliegue. Para reflejar la planeación completa hasta el entregable 6, se presenta una continuidad académica propuesta; las actividades descritas como planeación no deben considerarse evidencia de ejecución hasta que el equipo las valide.

Entregable	Enfoque	Evidencia que debe adjuntarse
1	Análisis y diseño	Anteproyecto, estructura, modelo ER y bocetos/interfaz inicial.
2	Lógica de negocio	Formularios, registro, autenticación, roles y flujo de solicitudes.
3	Integración inicial	Conexión PostgreSQL, publicación inicial y pruebas de rutas.
4	Mejora de experiencia de usuario	Comparativa visual antes/después, diseño responsive y consistencia CSS.
5	Integración funcional	Evidencia de formularios, persistencia, asignación, estados y reseñas.
6	Validación y liberación	Pruebas por rol, comprobación de nube, capturas de Render y verificación de /health.

5. Diseño de la solución

Interfaz y experiencia de usuario

La interfaz utiliza una paleta de azul marino, verde turquesa y fondos neutros, con jerarquía tipográfica, tarjetas de servicios, llamadas a la acción y estados visuales para las solicitudes. La hoja de estilos incluye reglas responsive y ajustes para movimiento reducido. El catálogo público presenta los servicios disponibles y precios iniciales de referencia.

Arquitectura

La aplicación se estructura con Flask para rutas y lógica del servidor, Jinja para las plantillas HTML, CSS para presentación y SQLAlchemy para el acceso a datos. PostgreSQL es la base objetivo en nube. Las variables sensibles se administran por entorno, no dentro del código fuente.

Roles

- Cliente: crea solicitudes y consulta su propio seguimiento; puede publicar una reseña cuando una solicitud está terminada.
- Especialista: consulta las solicitudes que tiene asignadas y actualiza el estado del servicio.
- Analista, gerente y administrador: consultan indicadores y gestionan asignaciones desde el dashboard.
- La autorización se valida en rutas del servidor; ocultar un enlace en la interfaz no sustituye la autorización.

6. Funcionalidades de la versión de mejora

- Inicio y catálogo público de plomería, electricidad y limpieza.
- Formulario de solicitud con campos obligatorios y validaciones básicas.
- Creación de cuenta de cliente en la primera solicitud y almacenamiento de contraseña mediante hash.
- Inicio y cierre de sesión con sesión de Flask.
- Portal de cliente con historial de solicitudes.
- Panel de especialista para actualizar estados de solicitudes asignadas.
- Dashboard con conteos, asignación manual de especialista y reseñas recientes.
- Ruta /health para comprobar la conexión a la base de datos.

Nota de alcance: este paquete de mejora no importa automáticamente el esquema ni los registros de la base existente. Antes de reemplazar una aplicación publicada, deben compararse los modelos, realizarse una copia de seguridad y planificarse las migraciones.

7. Seguridad y manejo de información

La contraseña no debe almacenarse en texto plano; la implementación usa las funciones de hash de Werkzeug. La clave de sesión y la cadena de conexión se configuran mediante variables de entorno. El archivo .env no debe subirse a GitHub. Los roles administrativos deben crearse por un procedimiento controlado, nunca mediante un formulario público. En producción se recomienda desactivar el modo debug, usar HTTPS, revisar logs, limitar permisos de base de datos y mantener copias de seguridad.

8. Despliegue en GitHub, Render y Aiven

GitHub

Se recomienda trabajar en una rama de mejora, conservar el historial y el remoto existentes, comprobar el estado de Git y excluir secretos mediante .gitignore. Antes de fusionar, se deben revisar los cambios y ejecutar las pruebas locales.

Render

El servicio web puede configurarse con el comando de construcción pip install -r requirements.txt y el comando de inicio gunicorn app:app. Las variables SECRET_KEY y DATABASE_URL se configuran en el panel del servicio. Después del despliegue, se debe revisar /health y los registros de ejecución.

Aiven

La cadena de conexión debe usar los valores de host, puerto, base, usuario y contraseña proporcionados por Aiven. Debe seguirse la configuración TLS/certificado CA indicada en el panel del proveedor. No se incluyen credenciales reales en este documento ni en el paquete.

9. Plan de pruebas y criterios de aceptación

Prueba	Criterio esperado
Inicio y catálogo	La página carga y se muestran los servicios.
Solicitud	Campos obligatorios validados y solicitud persistida.
Autenticación	Credenciales válidas abren sesión; las inválidas se rechazan.
Aislamiento de cliente	Un cliente solo consulta sus propias solicitudes.
Especialista	Solo ve solicitudes asignadas y puede cambiar estado permitido.
Asignación	Un usuario con permisos asigna un especialista existente.
Reseñas	Solo solicitudes terminadas admiten reseña y no se duplican.
Control por rol	Accesos no autorizados reciben respuesta 403.
Conexión nube	/health devuelve estado correcto y logs sin errores de conexión.
Diseño adaptable	Navegación y formularios usables en móvil y escritorio.

Las pruebas anteriores son una lista de verificación propuesta. Deben marcarse como aprobadas solo después de ejecutarlas y guardar capturas o resultados reproducibles.

10. Limitaciones y trabajo futuro

Quedan fuera de alcance los pagos en línea, las notificaciones por SMS/WhatsApp y la facturación electrónica. Antes de considerar la versión como liberada, se recomienda agregar pruebas automatizadas, migraciones formales de esquema, recuperación de contraseña, protección CSRF, límites de intentos de acceso, validación más estricta de fechas y datos, auditoría de cambios de estado y una política de privacidad.

11. Conclusiones

ServiHogar Durango puede organizar en una misma plataforma la presentación de servicios, la recepción de solicitudes y su seguimiento operativo. La mejora visual y la documentación técnica aportan una base más clara para continuar el desarrollo. La declaración de proyecto terminado debe sustentarse con pruebas ejecutadas, revisión de roles, comprobación de la conexión real a Aiven y evidencia de la aplicación desplegada en Render.

Referencias

Castañón Galván, E. A., Pulgarin Flores, Y. E., & Ramírez Campos, J. A. (2026). *Documentación del proyecto ServiHogar Durango: Relación con el anteproyecto y evidencia de los 3 entregables* [Documento de trabajo no publicado].

Pallets. (s. f.). *Flask documentation*. https://flask.palletsprojects.com/

Pallets. (s. f.). *Jinja documentation*. https://jinja.palletsprojects.com/

Render. (s. f.). *Web services*. https://render.com/docs/web-services

Aiven. (s. f.). *PostgreSQL*. https://aiven.io/docs/products/postgresql

SQLAlchemy. (s. f.). *SQLAlchemy documentation*. https://docs.sqlalchemy.org/